Multi-Agent Path Planning with Asymmetric Interactions In Tight Spaces Supplementary Material

VISMAY MODI, University of Toronto, Canada
YIXIN CHEN, University of Toronto, Canada
ABHISHEK MADAN, University of Toronto, Canada
SHINJIRO SUEDA, Texas A&M University, USA
DAVID I. W. LEVIN, University of Toronto, Canada

ACM Reference Format:
Vismay Modi, Yixin Chen, Abhishek Madan, Shinjiro Sueda, and David I. W. Levin. 2022. Multi-Agent Path Planning with Asymmetric Interactions In Tight Spaces Supplementary Material. 1, 1 (January 2022), 11 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 CONTROLLING AGENT BEHAVIOR

Controlling agent behavior is very intuitive in our method. There are several points of control in our simulation. First, each agent has its own pre-determined characteristics profile. This includes things such as size, mass, stubbornness, collision radius, etc... Each of these parameters can be intuitively adjusted for any agent to get the desired behavior. Another control point for this method is adjusting the coefficients of the energy. If the scene happens to be on a large, unconstrained map, its advantageous to turn $K^M = 0$ off to prevent possible numerical issues from the smooth distance function. Agents are allowed to collide if K^C is set to 0. The grouping cost can be modified through the grouping parameter for each agent pair K^G. If agents need to reach their goal by a specific end time, we use K^T to control this behavior. Acceleration is controlled by the K_A parameter. Regularizing the edge lengths of each curve is important, as explained above, but modifying K^R can affect the likelihood of the solver getting stuck in local minimums in highly constrained environments. Last, but not least, K^K weights the kinetic energy term and increasing it (or increasing the mass term m) will incentivize the agent to follow a more direct path to the end location. It is very intuitive to play with these parameters in order to get the desired behavior in your multi-agent simulation scene.

1.1 Sparsifying Interactions

We should note that a loop over all agents is not strictly necessary for the interaction costs. It is possible to control the agent-agent interactions by either manually denoting agents that will interact with each other or algorithmically sparseifying interactions for agents whose trajectories are far away using a BVH. In our examples we present worst-case

Authors' addresses: Vismay Modi, vismay@cs.toronto.edu, University of Toronto, Canada; Yixin Chen, yixinc.chen@mail.utoronto.ca, University of Toronto, Canada; Abhishek Madan, amadan@cs.toronto.edu, University of Toronto, Canada; Shinjiro Sueda, sueda@tamu.edu, Texas A&M University, USA; David I. W. Levin, diwlevin@cs.toronto.edu, University of Toronto, Canada.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2022 Association for Computing Machinery.
Manuscript submitted to ACM

performance stress-tests where every agent in the scene interacts with every other agent; however for the grouping scenes, the three agent pond scenes and the corn maze, we manually sparsify some of the interaction costs for an extra performance boost.

1.2 Cost Function Pseudo-code

Algorithm 1 The cost function is a summation of all individual costs. This is used in the interior point solve.

Require: global input variables from main paper
Require: @ALPHAHEURISTIC (main paper)
  function COSTS($\mathbf{S}, \alpha_0, K, R, T, M, b_v, T^{max}, \mu$)
    $f \leftarrow 0$
    for $a, b \epsilon (0, .., \|A\|)$ **do**
      $\mathbf{s}_a, \mathbf{s}_b$ from $\mathbf{S}$
      r_a, r_b from R
      K^C, K^G from K
      $\alpha \leftarrow ALPHAHEURISTIC(\alpha_0, \mathbf{s}_a, \mathbf{s}_b)$
      $f \leftarrow f + \mu C^C(\mathrm{d}(\alpha, \mathbf{s}_a, \mathbf{s}_b), K^C, r^a, r^b)$
      $f \leftarrow f + \mu C^G(\mathrm{d}(\alpha, \mathbf{s}_a, \mathbf{s}_b), K^G, r^a, r^b)$
    end for
    for $a \epsilon (0, .., \|A\|)$ **do**
      $\alpha \leftarrow ALPHAHEURISTIC(\alpha_0, \mathbf{s}_a, b_v)$
      $\mathbf{s}_a, r_a, m_a, T_a^p, T_a^{max}$ from $\mathbf{S}, R, M, T, T^{max}$
      K^K, K^A, K^T, K^R, K^M from K
      $f \leftarrow f + C^M(\mathrm{d}(\alpha, \mathbf{s}_a, b_v), K^M, r^a)$
      $f \leftarrow f + C^K(\mathbf{s}_a, m_a, K^K) + C^A(\mathbf{s}_a, K^A)$
      $f \leftarrow f + C^T(\mathbf{s}_a, K^T, T_a^p) + C^R(\mathbf{s}_a, K^R)$
    end for
    return f
  end function

2 DISCRETE GRADIENTS

Included below are the discrete gradients for each of our cost terms. For the sake of brevity, we do not include the gradient assembly/aggregation steps, only the element-wise gradient terms.

2.1 Kinetic Gradient

The kinetic energy gradient below is aggregated over each trajectory curve segment $e^i = [x^i, y^i, t^i, x^{i+1}, y^{i+1}, t^{i+1}]$ for each agent in the entire scene.

$$\nabla_{x^i} C^K(\mathbf{s}_a) = -K_a^K m_a \frac{x^{i+1} - x^i}{t^{i+1} - t^i} \tag{1}$$

$$\nabla_{y^i} C^K(\mathbf{s}_a) = -K_a^K m_a \frac{y^{i+1} - y^i}{t^{i+1} - t^i} \tag{2}$$

$$\nabla_{t^i} C^K(\mathbf{s}_a) = \frac{1}{2} K_a^K m_a \frac{(x^{i+1} - x^i)^2 + (y^{i+1} - y^i)^2}{(t^{i+1} - t^i)^2} \tag{3}$$

$$\nabla_{x^{i+1}} C^K(\mathbf{s}_a) = -\nabla_{x^i} C^K(\mathbf{s}_a) \tag{4}$$

$$\nabla_{y^{i+1}} C^K(\mathbf{s}_a) = -\nabla_{y^i} C^K(\mathbf{s}_a) \tag{5}$$

$$\nabla_{t^{i+1}} C^K(\mathbf{s}_a) = -\nabla_{t^i} C^K(\mathbf{s}_a) \tag{6}$$

$$\tag{7}$$

2.2 Acceleration Gradient

The accelation energy measures the bend between two rod segments e^{i-1} and e^i. The gradient is aggregated over the non-boundary nodes of each trajectory curve $\mathbf{n}^i = [x^i, y^i, t^i]$ where $i \neq 0, n$ for an agent in the scene. Using the chain rule, we get

$$\nabla_{\mathbf{n}^{i-1}} C^A(\mathbf{s}_a) = \frac{\partial C^A}{\partial \theta^i} \frac{\partial \theta^i}{\partial n^{i-1}} \tag{8}$$

$$\nabla_{\mathbf{n}^i} C^A(\mathbf{s}_a) = \frac{\partial C^A}{\partial \theta^i} \frac{\partial \theta^i}{\partial n^i} \tag{9}$$

$$\nabla_{\mathbf{n}^{i+1}} C^A(\mathbf{s}_a) = \frac{\partial C^A}{\partial \theta^i} \frac{\partial \theta^i}{\partial n^{i+1}} \tag{10}$$

After further chain rule and breaking down each partial into its individual components, we get

$$v^i = \mathbf{n}^{i+1} - \mathbf{n}^i \tag{11}$$

$$v^{i+1} = \mathbf{n}^i - \mathbf{n}^{i-1} \tag{12}$$

$$\nabla_{\mathbf{n}^{i-1}} C^K(\mathbf{s}_a) = -K^A \theta^i \left(\frac{1}{1 + (\frac{\|v^i \times v^{i+1}\|}{v^i \cdot v^{i+1}})} \right) \left(\frac{v^i \cdot v^{i+1} \frac{1}{\|v^i \times v^{i+1}\|} * \left((v^i \times v^{i+1})^T [v^{i+1}]_{\mathrm{M}} \right) - v^{i+1} \|v^i \times v^{i+1}\|}{(v^i \cdot v^{i+1})^2} \right) \tag{13}$$

$$\nabla_{\mathbf{n}^{i+1}} C^K(\mathbf{s}_a) = K^A \theta^i \left(\frac{1}{1 + (\frac{\|v^i \times v^{i+1}\|}{v^i \cdot v^{i+1}})} \right) \left(\frac{v^i \cdot v^{i+1} \frac{1}{\|v^i \times v^{i+1}\|} * \left((v^i \times v^{i+1})^T [v^i]_{\mathrm{M}} \right) - v^i \|v^i \times v^{i+1}\|}{(v^i \cdot v^{i+1})^2} \right) \tag{14}$$

$$\nabla_{\mathbf{n}^i} C^K(\mathbf{s}_a) = -\nabla_{\mathbf{n}^{i-1}} C^K(\mathbf{s}_a) - \nabla_{\mathbf{n}^{i+1}} C^K(\mathbf{s}_a) \tag{15}$$

In the acceleration gradient, we manually set $\nabla_{t^{i-1}} C^K(\mathbf{s}_a) = \nabla_{t^i} C^K(\mathbf{s}_a) = \nabla_{t^{i+1}} C^K(\mathbf{s}_a) = 0$ since the kinetic gradient already controls acceleration in the t direction.

2.3 Preferred End Time Gradient

This gradient only affects the t^n for an agent as shown below

$$\nabla_{t^n} C^T(\mathbf{s}_a) = K^T(t^n - T^p) \tag{16}$$

2.4 Regularizer Gradient

The regularizer gradient below is aggregated over each trajectory curve segment $e^i = [x^i, y^i, t^i, x^{i+1}, y^{i+1}, t^{i+1}]$ for each agent in the entire scene. The regularizer energy and by extension, gradient only regularize time intervals over the trajectory as follows:

$$\nabla_{t^i} C^R(\mathbf{s}_a) = K^T \frac{\frac{t^n}{n}}{(t^{i+1} - t^i)^2} \tag{17}$$

$$\nabla_{t^{i+1}} C^R(\mathbf{s}_a) = -K^T \frac{\frac{t^n}{n}}{(t^{i+1} - t^i)^2}. \tag{18}$$

2.5 Collision Interaction Gradient

The collision gradient for the interaction between agent a and agent b is aggregated over each curve segment for the two trajectories. First we take the derivatives of the LogSumExp smooth distance function $\mathrm{d}(\alpha, \mathrm{u}(\mathbf{s}_a), \mathrm{u}(\mathbf{s}_b))$ w.r.t to the upsampled trajectory curves of size $m >> n$ $\mathrm{u}(\mathbf{s}) = \mathrm{upsample}(\mathbf{s})$ with $k, q \epsilon 0, .., m$ denoting the upsampled node indices.

$$\frac{\partial \mathrm{d}}{\partial \mathrm{u}_a^k} = \frac{-1}{\alpha} e^{-\alpha \|\mathrm{u}_a^k - \mathrm{u}_b^q\|} \frac{\mathrm{u}_a^k}{\|\mathrm{u}_a^k - \mathrm{u}_b^q\|} \tag{19}$$

$$\frac{\partial \mathrm{d}}{\partial \mathrm{u}_b^q} = \frac{-1}{\alpha} e^{-\alpha \|\mathrm{u}_a^k - \mathrm{u}_b^q\|} \frac{-\mathrm{u}_b^q}{\|\mathrm{u}_a^k - \mathrm{u}_b^q\|} \tag{20}$$

Next, we put the LogSumExp derivatives together with the $m \times n$ jacobians $J_a = \frac{\partial \mathrm{u}_a}{\partial \mathbf{s}_a}$ and $J_b = \frac{\partial \mathrm{u}_b}{\partial \mathbf{s}_b}$ into the gradients

$$\nabla_{\mathbf{s}_a} C^C(\mathrm{d}(\alpha, \mathrm{u}(\mathbf{s}_a), \mathrm{u}(\mathbf{s}_b)), r_a, r_b) = -\frac{K_a^C}{-r_a - r_b + \mathrm{d}} \frac{\partial \mathrm{d}}{\partial \mathrm{u}_a^k} J_a \tag{21}$$

$$\nabla_{\mathbf{s}_b} C^C(\mathrm{d}(\alpha, \mathbf{s}_a, \mathbf{s}_b), r_a, r_b) = -\frac{K_b^C}{-r_a - r_b + \mathrm{d}} \frac{\partial \mathrm{d}}{\partial \mathrm{u}_b^k} J_b \tag{22}$$

Summed up over each agent-agent interaction interaction in the scene, this gives us the collision gradient.

2.6 Grouping(Friendship) Interaction Gradient

We use the same notation and jacobians ($\frac{\partial u}{\partial \mathbf{s}}$) as the Collision Gradient above to describe the Grouping cost's gradient

$$\nabla_{\mathbf{s}_a} C^G(\mathrm{d}(\alpha, \mathbf{s}_a, \mathbf{s}_b), r_a, r_b) = 2K_a^G(-r_a - r_b + \mathrm{d}) \frac{\partial \mathrm{d}}{\partial \mathrm{u}_a^k} J_a \tag{23}$$

$$\nabla_{\mathbf{s}_b} C^G(\mathrm{d}(\alpha, \mathbf{s}_a, \mathbf{s}_b), r_a, r_b) = 2K_b^G(-r_a - r_b + \mathrm{d}) \frac{\partial \mathrm{d}}{\partial \mathrm{u}_b^k} J_b \tag{24}$$

2.7 Map Interaction Gradient

For interactions with the map boundary vertices b_v, we calculate the gradients using similar notation as above, except since the boundary is 2D, we can zero out the t dimension before passing points into the LogSumExp distance function. The map gradient for agent a is

$$\nabla_{\mathbf{s}_a} C_a^M(\mathrm{d}(\alpha, \mathbf{s}_a, b_v), r_a,) = \frac{K_a^M}{(-r_a + \mathrm{d})} \frac{\partial \mathrm{d}}{\partial \mathrm{u}_a^k} J_a \tag{25}$$

$$\tag{26}$$

2.8 Gradient Pseudocode

Algorithm 2 The gradient is assembled as a sump of the individual components. In the appendix we include the gradient formulations. The resulting gradient is used in the interior point (IPSover).

Require: global input variables from Algorithm **??**
Require: @ALPHAHEURISTIC (**??**)
  function GRADS($\mathbf{S}, \alpha_0, K, R, T, M, b_v, T^{max}, \mu$)
    $g \leftarrow \mathbf{0}$
    for $a, b \epsilon (0, .., \|A\|)$ **do**
      $\mathbf{s}_a, \mathbf{s}_b$ from $\mathbf{S}$
      r_a, r_b from R
      K^C, K^G from K
      $\alpha \leftarrow ALPHAHEURISTIC(\alpha_0, \mathbf{s}_a, \mathbf{s}_b)$
      $g \leftarrow g + \mu \nabla C^C(\mathrm{d}(\alpha, \mathbf{s}_a, \mathbf{s}_b), K^C, r^a, r^b)$
      $g \leftarrow g + \mu \nabla C^G(\mathrm{d}(\alpha, \mathbf{s}_a, \mathbf{s}_b), K^G, r^a, r^b)$
    end for
    for $a \epsilon (0, .., \|A\|)$ **do**
      $\alpha \leftarrow ALPHAHEURISTIC(\alpha_0, \mathbf{s}_a, b_v)$
      $\mathbf{s}_a, r_a, m_a, T_a^p, T_a^{max}$ from $\mathbf{S}, R, M, T, T^{max}$
      K^K, K^A, K^T, K^R, K^M from K
      $g \leftarrow g + \nabla C^M(\mathrm{d}(\alpha, \mathbf{s}_a, b_v), K^M, r^a)$
      $g \leftarrow g + \nabla C^K(\mathbf{s}_a, m_a, K^K) + \nabla C^A(\mathbf{s}_a, K^A)$
      $g \leftarrow g + \nabla C^T(\mathbf{s}_a, K^T, T_a^p) + \nabla C^R(\mathbf{s}_a, K^R)$
    end for
    return g
  end function

3 DISCRETE HESSIANS AND APPROXIMATIONS

3.1 Kinetic Hessian

The kinetic hessian below is aggregated over each trajectory curve segment $e^i = [x^i, y^i, t^i, x^{i+1}, y^{i+1}, t^{i+1}]$ for each agent $a = [1, \|A\|]$ in the entire scene. The element-wise block is

$$m_a \begin{bmatrix} \frac{1}{(t^{i+1}-t^i)} & 0 & 0 & -\frac{1}{(t^{i+1}-t^i)} & 0 & 0 \\ 0 & \frac{1}{(t^{i+1}-t^i)} & 0 & 0 & -\frac{1}{(t^{i+1}-t^i)} & 0 \\ 0 & 0 & \frac{(x^{i+1}-x^i)^2+(y^{i+1}-y^i)^2}{(t^{i+1}-t^i)^3} & 0 & 0 & -\frac{(x^{i+1}-x^i)^2+(y^{i+1}-y^i)^2}{(t^{i+1}-t^i)^3} \\ -\frac{1}{(t^{i+1}-t^i)} & 0 & 0 & \frac{1}{(t^{i+1}-t^i)} & 0 & 0 \\ 0 & -\frac{1}{(t^{i+1}-t^i)} & 0 & 0 & \frac{1}{(t^{i+1}-t^i)} & 0 \\ 0 & 0 & -\frac{(x^{i+1}-x^i)^2+(y^{i+1}-y^i)^2}{(t^{i+1}-t^i)^3} & 0 & 0 & \frac{(x^{i+1}-x^i)^2+(y^{i+1}-y^i)^2}{(t^{i+1}-t^i)^3} \end{bmatrix} \tag{27}$$

3.2 Acceleration Hessian

The acceleration hessian measures the second order effects of the bend between two rod segments e^{i-1} and e^i in the trajectory curve of agent a. It is aggregated over the inside nodes, $\mathbf{n}^i = [x^i, y^i, t^i]$ where $i \neq 0, n$, of the trajectory curve. WLOG, for the non-boundary node i, we subdivide the symmetric $9x9$ hessian into the following unique $3x3$ blocks

$$\frac{\partial^2 C_a^A}{\partial n^{i-1} \partial n^{i-1}} = H^{1,1} \tag{28}$$

$$\frac{\partial^2 C_a^A}{\partial n^{i-1} \partial n^{i+1}} = H^{1,3} \tag{29}$$

$$\frac{\partial^2 C_a^A}{\partial n^{i+1} \partial n^{i+1}} = H^{3,3}. \tag{30}$$

The indexes of the block correspond to their placement in the $9x9$ hessian. We put together the node-wise hessian as

$$\begin{bmatrix} H^{1,1} & -H^{1,1} - H^{1,3} & H^{1,3} \\ (-H^{1,1} - H^{1,3})^T & -(-H^{1,1} - H^{1,3})^T - (-(H^{1,3})^T - H^{3,3})^T & (-(H^{1,3})^T - H^{3,3})^T \\ (H^{1,3})^T & -(H^{1,3})^T - H^{3,3} & H^{3,3} \end{bmatrix}. \tag{31}$$

For brevity, we define the following terms for use in just the acceleration hessian section:

$$d = v^i \cdot v^{i+1} \tag{32}$$

$$n = v^i \times v^{i+1} \tag{33}$$

$$c = \|v^i \times v^{i+1}\| \tag{34}$$

$$q = \text{atan2}(n, d) \tag{35}$$

$$m = \frac{1}{1 + \left(\frac{n}{d}\right)^2} \tag{36}$$

$$L = [v^i]_{\text{M}} \text{ cross product matrix of } v^i \tag{37}$$

$$M = [v^{i+1}]_{\text{M}} \text{ cross product matrix of } v^{i+1} \tag{38}$$

$$T = (M \otimes v^{i+1})_\gamma c^\gamma \text{ tensor contraction} \tag{39}$$

The three unique components of the hessian are

$$H^{1,1} = -m\left(-m\frac{\frac{d}{n}(c)^T[v^{i+1}]_{\mathrm{M}} - v^{i+1}n}{d^2}\right)\left(\frac{\frac{d}{n}(c)^T[v^{i+1}]_{\mathrm{M}} - v^{i+1}n}{d^2}\right)^T$$
$$+ -q\left(\left(\frac{2\frac{n}{d}}{m^2}\right)\frac{\frac{d}{n}(c)^T[v^{i+1}]_{\mathrm{M}} - v^{i+1}n}{d^2}\right)\left(\frac{\frac{d}{n}(c)^T[v^{i+1}]_{\mathrm{M}} - v^{i+1}n}{d^2}\right)^T \tag{40}$$
$$+ -qm\left(\frac{d^2n((Mc(v^{i+1})^T - d(M)^TM) + (v^{i+1}2c^TM)) - (dc^TM - (v^{i+1})^Tn^2) * (-2dnv^{i+1} + -(d^2(\frac{1}{n})(c^TM)))^T}{(d^2n)^2}\right),$$

$$H^{1,3} = -m\left(m\frac{\frac{d}{n}(c)^T[v^{i}]_{\mathrm{M}} - v^{i}n}{d^2}\right)\left(\frac{\frac{d}{n}(c)^T[v^{i+1}]_{\mathrm{M}} - v^{i+1}n}{d^2}\right)^T \tag{41}$$
$$+ -q\left(\left(-\frac{2\frac{n}{d}}{m^2}\right)\frac{\frac{d}{n}(c)^T[v^{i}]_{\mathrm{M}} - v^{i}n}{d^2}\right)\left(\frac{\frac{d}{n}(c)^T[v^{i+1}]_{\mathrm{M}} - v^{i+1}n}{d^2}\right)^T \tag{42}$$
$$+ -qm\left(\frac{d^2n(((M^Tc)(v^i)^T - d(M^TL^T + T)) + -(\mathrm{I}n^2 + 2v^{i+1}c^TL)) - (dc^TM - (v^{i+1})^Tn^2)((2dnv^i) + (d^2\frac{1}{n}(c^TL)))^T}{(d^2n)^2}\right), \tag{43}$$

$$H^{3,3} = m\left(m\frac{\frac{d}{n}(c)^T[v^{i}]_{\mathrm{M}} - v^{i}n}{d^2}\right)\left(\frac{\frac{d}{n}(c)^T[v^{i}]_{\mathrm{M}} - v^{i}n}{d^2}\right)^T \tag{44}$$
$$+ q\left(\left(-\frac{2\frac{n}{d}}{m^2}\right)\frac{\frac{d}{n}(c)^T[v^{i}]_{\mathrm{M}} - v^{i}n}{d^2}\right)\left(\frac{\frac{d}{n}(c)^T[v^{i}]_{\mathrm{M}} - v^{i}n}{d^2}\right)^T \tag{45}$$
$$+ qm\left(\frac{d^2n(((L^Tc)(v^i)^T - dL^TL^T) + -(v^i2c^TL)) - (dc^TL - (v^i)^Tn^2)((2dnv^i) + (d^2\frac{1}{n}(c^TL)))^T}{(d^2n)^2}\right). \tag{46}$$

The resulting $9x9$ hessian is assembled over each interior node of each trajectory curve in the scene.

3.3 Preferred End Time Hessian

This is the simplest hessian. It is the constant matrix

$$\begin{bmatrix} 0 & & \\ & \ddots & 0 \\ & 0 & 1 \end{bmatrix} \tag{47}$$

for each agent in the scene since the only non-zero value in the hessian is $\frac{\partial^2 C_a^T}{\partial t^n \partial t^n}$

3.4 Regularizer Approximate Hessian

The regularizer hessian below is aggregated over each trajectory curve segment $e^i = [x^i, y^i, t^i, x^{i+1}, y^{i+1}, t^{i+1}]$ for each agent in the entire scene in a symmetric block-diagonal fashion. The approximation assumes the end time, t^n, for each agent is constant. Therefore, for each trajectory curve segment in the scene, we assemble to following block into the global hessian:

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 2K_a^R \frac{\frac{t^n}{n}}{(t^{i+1}-t^i)^3} & 0 & 0 & -2K_a^R \frac{\frac{t^n}{n}}{(t^{i+1}-t^i)^3} \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & -2K_a^R \frac{\frac{t^n}{n}}{(t^{i+1}-t^i)^3} & 0 & 0 & 2K_a^R \frac{\frac{t^n}{n}}{(t^{i+1}-t^i)^3} \end{bmatrix} \tag{48}$$

3.5 Collision Interaction Approximate Hessian

The collision hessian approximation is an agent-wise block diagonal hessian with dense individual blocks. Each block is subsequently sparsified by the function in 3. The agent-wisedense approximation is the following

$$\nabla^2_{\mathbf{s}_a} C^C(\mathrm{d}(\alpha, \mathrm{u}(\mathbf{s}_a), \mathrm{u}(\mathbf{s}_b)), r_a, r_b) = \frac{K_a^C}{(-r_a - r_b + \mathrm{d})^2} \left(\frac{\partial \mathrm{d}}{\partial \mathrm{u}_a^k} J_a \right) \left(\frac{\partial \mathrm{d}}{\partial \mathrm{u}_a^k} J_a \right)^T . \tag{49}$$

This term is passed into the sparsification function which uses the kronecker product to zero out the indices of the hessian which correspond to vertices that have an absolute distance greater than the given cutoff value. The sparse hessian is subsequently assembled into the correct block of the overall hessian as shown in Figure 1. For simplicity, we

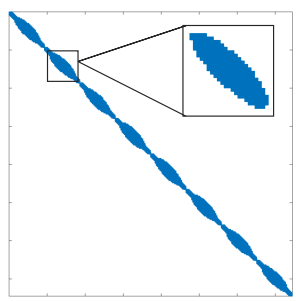

Fig. 1. This is the assembled collision hessian matrix approximation where each block diagonal component has been passed through a sparsification function.

drop the smoothing function's 2nd derivative from our hessian and drop the off-diagonal terms. We find that dropping the off-diagonal blocks actually improves convergence and smoothness of trajectories.

3.6 Grouping(Friendship) Interaction Hessian

Using the same notation as in the collision interaction hessiani, we present the grouping hessian. This is passed through the same sparsification function as the collision hessian before assembly.

$$\nabla^2_{\mathbf{s}_a} C^G(\mathrm{d}(\alpha, \mathrm{u}(\mathbf{s}_a), \mathrm{u}(\mathbf{s}_b)), r_a, r_b) = 2K_a^G \left(\frac{\partial \mathrm{d}}{\partial \mathrm{u}_a^k} J_a \right) \left(\frac{\partial \mathrm{d}}{\partial \mathrm{u}_a^k} J_a \right)^T \tag{50}$$

3.7 Map Interaction Approximate Hessian

Again, we pass the following map interaction hessian through a sparsification function and then assemble it together into the global hessian.

$$\nabla^2_{\mathbf{s}_a} C^M(\mathrm{d}(\alpha, \mathrm{u}(\mathbf{s}_a), b_v), r_a, r_b) = \frac{K_a^M}{(-r_a + \mathrm{d})^2} \left(\frac{\partial \mathrm{d}}{\partial \mathrm{u}_a^k} J_a \right) \left(\frac{\partial \mathrm{d}}{\partial \mathrm{u}_a^k} J_a \right)^T . \tag{51}$$

3.8 Hessian and Hessian Appx Pseudo-code

Algorithm 3 The hessian is assembled as a sump of the individual components. In the appendix we include the hessian formulations and approximations. Some of the components, H_C, H_M, are block diagonal, but have dense blocks. So we sparsify these blocks using the SPARSIFYHESS function. In the end, we have a sparse hessian to use in the interior point (IPSover).

Require: : global input variables from Algorithm **??**
Require: @ALPHAHEURISTIC (**??**)

```
function HESS(S, α_0, K, R, T, M, b_v, T^max, μ)
    H, H_M, H_G, H_C ← 0
    for a, b ϵ (0, .., ||A||) do
        s_a, s_b from S
        r_a, r_b from R
        K^C, K^G from K
        α ← ALPHAHEURISTIC(α_0, s_a, s_b)
        H_C ← μ∇²C^C(d(α, s_a, s_b), K^C, r^a, r^b)
        H_G ← μ∇²C^G(d(α, s_a, s_b), K^G, r^a, r^b)
        H ← H + SPARSIFYHESS(H_C + H_G, s_a, s_b, cutoff)
    end for
    for a ϵ (0, .., ||A||) do
        α ← ALPHAHEURISTIC(α_0, s_a, b_v)
        s_a, r_a, m_a, T_a^p, T_a^max from S, R, M, T, T^max
        K^K, K^A, K^T, K^R, K^M from K
        H_M ← ∇²C^M(α, s_a, b_v, K^M, r^a)
        H ← H + SPARSIFYHESS(H_M, s_a, b_v, cutoff)
        H ← H + ∇²C^K(s_a, m_a, K^K) + ∇²C^A(s_a, K^A)
        H ← H + ∇²C^R(s_a, K^R)
    end for
    return H
end function

function SPARSIFYHESS(H, V_1, V_2, cutoff)
    D ← maxAbsoluteDist(V_1, V_2)
    for i ϵ [0, rows(H)] do
        for j ϵ [0, cols(H)] do
            if H(i, j) < cutoffD then
                H(i, j) ← 0
            end if
        end for
    end for
end function
```

4 TABLE OF ALL EXAMPLES

Scene	Num Agents	DOFs	Layers	Djikstra Time(s) Per Agent	Avg Time(s) Per Iteration	Total Solve Time(s)	Num OuterIterations
2 Circle	2	100	11	0.217	0.165	1.153	5
8 Circle	8	400	11	0.203	2.252	49.993	5
10 Circle	10	500	11	0.200	2.705	422.125	5
20 Circle	20	1000	11	0.193	10.474	1425.874	5
30 Circle	30	1500	9	0.165	24.146	1959.703	5
Airplane Aisle	5	250	10	0.788	0.130	37.031	1
3 Colliding Agents	3	90	5	0.131	0.012	0.456	2

3 Symmetric Agents	3	90	5	0.134	0.028	2.355	2
Snake and People	3	90	5	0.134	0.050	3.820	5
Deer and People	3	90	5	0.129	0.062	2.671	5
Elephant and People	3	90	5	0.131	0.012	0.346	1
Grouping Friends	3	90	5	0.129	0.021	0.849	2
Robot intersection	3	105	10	0.131	0.015	2.048	1
Elphant Bottleneck	5	250	5	0.299	0.107	29.607	1
Corn Maze	5	1000	3	6.343	1.095	249.750	3
Circle Maze	4	1200	3	1.175	2.877	385.526	1
Subway Platform	2	100	10	0.083	0.036	3.177	2
Elephant Field	83	16 600	2	11.947	634.642	13 962.124	1
Safari	12	660	15	1.307	1.220	79.729	1
Smily Face	24	1080	20	4.000	4.497	535.118	1

Table 1. Table of info for all examples